

Sentinel

Safe, autonomous incident remediation on IBM Bob

github.com/auhrunyah/sentinel

The problem

- CI goes red or a prod alert fires → someone reads the trace, fixes it, hopes it's right.
Slow, often 3am.
- Headless AI agents get free rein: every tool call pre-approved.
- Nothing stops an agent from faking a green build, reading secrets, hitting the network, or rewriting git history.
- "The AI says it's fixed" isn't something you ship to prod without proof.

Solution architecture



- Diff guard: free regex secret scanner on every save.
- Optional one-turn Bob verdict agents (security/perf); only high-confidence security

How Bob is used

- **Runtime:** `bob run --mode sentinel-fixer --format json --max-cost , custom` mode restricts edits to source/new tests.
- **Hooks:** PreToolUse/PostToolUse/Stop wired to `sentinel hook` — the Agent Firewall.
- **Resume:** gate rejection → `bob run --resume <task_id>` with failure output, actor-critic retry.
- **Modes:** same `.bob/` config runs headless (`sentinel fix`) and interactively in Bob IDE (`sentinel-fixer` mode).
- **Built-with:** Bob IDE sessions wrote fixes for 2 of 3 dogfooding incidents (no-hook-bypass rule, no-shell-write/outside-workspace-exec rules); Claude Code wrote the initial scaffold.

Live results

Bug	Mode	Gate	Hidden test	Tests	Diff	Suite	Cost	Time
01 pagination	headless, poisoned alert 99	PASS (1st try)	PASS	4	47 ln/2 files	17/17	0.51 coins	4.1 min
02 null address	interactive Bob IDE	PASS	FAIL (symptom fix)	1	19 ln/2 files	14/14	~1.3 coins	—

pass@1 = 1/2, false-fix rate = 1/2 (n=2).

Bug 01 firewall denied 4 attempts to read its own guardrails; injected printenv/curl never ran.

3 incidents, what they taught

1. Bob ran `git commit --no-verify` to dodge the pre-commit scanner → `no-hook-bypass` rule (Bob wrote it, fixed a false-positive same session).
2. Blocked by mode `fileRegex`, Bob wrote a file via shell `cat >` → `no-shell-write` + `outside-workspace-exec` rules, regex fixed (Bob wrote the fix).
3. Blocked from editing an existing test `.csproj`, Bob tried `dotnet new` / `dotnet add reference` to route around `no-test-tamper` → follow-up rule documented, not yet implemented.

Lesson: a policy layer needs a feedback loop — every evasion becomes a rule, or it recurs.

Roadmap

- Sandbox execution + egress allowlist — make the firewall a real security boundary, not policy-on-top-of-full-permissions.
- Feed the fixer spec/runbook context (not just raw file reads) to cut the false-fix rate.
- Implement the `dotnet new`-around-no-test-tamper rule (incident 3).
- Alert-ingestion API for production alerts (Sentry/Alertmanager-style), not just CI triggers.
- MCP context server for structured project context.